Lesson 4 of 5

The growing tail

Each meal adds a tail segment that follows the head.

Goal	Each time the snake eats, it grows a segment that follows the head.
Concepts	Lists, append, for loops, creating objects while the program runs.
Starting point	Lesson 3's finished code. The complete Lesson 4 code is on the last page.
Teacher prep	Clear a space for the human-snake activity (4 volunteers).

Session plan (30 minutes)

Time	Activity
0-5	Run Lesson 3. Ask: why does the snake not grow?
5-12	Human snake activity (see teaching notes). This is the lesson's hardest idea, so give it time.
12-25	Students add the code and test.
25-30	Challenge and wrap-up.

Step 1: a list for the tail

Add this **after the food setup**:

```
segments = []
```

Step 2: two new functions

Add these **above game_loop**:

```
def add_segment():
    seg = turtle.Turtle("square")
    seg.color("green")
    seg.penup()
    segments.append(seg)

def move_body():
    # Start at the tail so each segment copies the one in front of it
    for i in range(len(segments) - 1, 0, -1):
        segments[i].goto(segments[i - 1].pos())
    if segments:
        segments[0].goto(head.pos())
```

Step 3: update game_loop

```
    if head.distance(food) < STEP:
        move_food()
        add_segment()          # new line

    move_body()                # new line, goes BEFORE move_head()
    move_head()
```

Step 4: stop the snake reversing into itself

Now there is a tail, replace the four steering functions:

```
def go_up():
    if head.direction != "down": head.direction = "up"
```

```
def go_down():
    if head.direction != "up": head.direction = "down"
def go_left():
    if head.direction != "right": head.direction = "left"
def go_right():
    if head.direction != "left": head.direction = "right"
```

Teaching notes: the human snake

Line up four students as a snake. The head steps forward. If the person right behind moves into the head's **old** spot first, then the next person moves into theirs, everything works. If you start from the front instead, everyone ends up on the same square. Act it out both ways, then show the backwards loop: **range(len(segments) - 1, 0, -1)** counts from the tail down to 1.

Heads-up: if the snake hits a wall now, the tail stays behind. That is deliberate. Lesson 5 fixes it with reset().

Challenge

- Make each new segment a different colour.
- Print len(segments) in the shell each time the snake eats.
- Stretch: make the head a different shape from the tail.

Common problems

What you see	Likely cause and fix
The snake never grows	add_segment() is not called inside the if head.distance(food) block.
The tail lags or jumps oddly	move_body() must come **before** move_head().
The tail is a clump of squares in the middle	The loop's range is wrong. It must be range(len(segments) - 1, 0, -1).
Pressing the opposite arrow does nothing	That is the new behaviour from Step 4.

What we learned (student checklist)

[ ] I can store many objects in a list and add to it with append.

[ ] I can explain why the tail moves from the back to the front.

[ ] I can create new turtles while the program is running.

[ ] I can use a for loop with range.

Complete Lesson 4 code (for catching up)

```
import turtle
import random

STEP = 20
LIMIT = 280

screen = turtle.Screen()
screen.title("Snake")
screen.bgcolor("black")
screen.setup(600, 600)
screen.tracer(0)

head = turtle.Turtle("square")
head.color("lime")
head.penup()
head.direction = "stop"

food = turtle.Turtle("circle")
food.color("red")
food.penup()
food.goto(0, 100)

segments = []

def go_up():
    if head.direction != "down": head.direction = "up"
def go_down():
    if head.direction != "up": head.direction = "down"
def go_left():
    if head.direction != "right": head.direction = "left"
def go_right():
    if head.direction != "left": head.direction = "right"

def move_head():
    if head.direction == "up":    head.sety(head.ycor() + STEP)
    if head.direction == "down":  head.sety(head.ycor() - STEP)
    if head.direction == "left":  head.setx(head.xcor() - STEP)
    if head.direction == "right": head.setx(head.xcor() + STEP)

def move_body():
    # Start at the tail so each segment copies the one in front of it
    for i in range(len(segments) - 1, 0, -1):
        segments[i].goto(segments[i - 1].pos())
    if segments:
        segments[0].goto(head.pos())

def add_segment():
    seg = turtle.Turtle("square")
    seg.color("green")
    seg.penup()
    segments.append(seg)

def move_food():
    food.goto(random.randrange(-LIMIT, LIMIT, STEP),
              random.randrange(-LIMIT, LIMIT, STEP))

def game_loop():
    if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
        head.goto(0, 0)
        head.direction = "stop"

    if head.distance(food) < STEP:
        move_food()
        add_segment()
```

```
    move_body()
    move_head()
    screen.update()
    screen.ontimer(game_loop, 100)

screen.listen()
screen.onkeypress(go_up, "Up")
screen.onkeypress(go_down, "Down")
screen.onkeypress(go_left, "Left")
screen.onkeypress(go_right, "Right")

game_loop()
screen.mainloop()
```